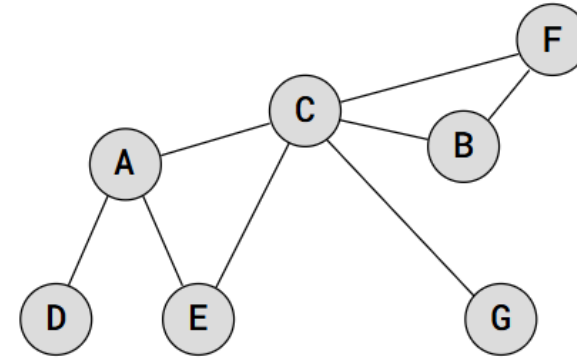
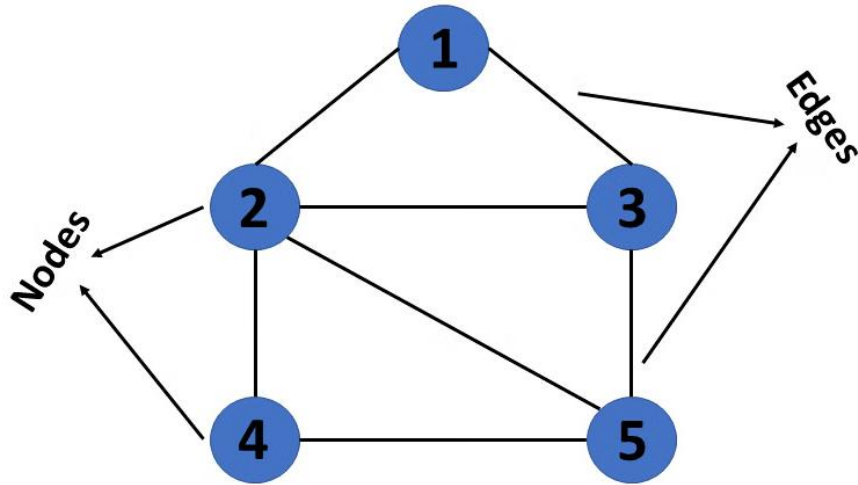


Module 5 - Graphs

Terminology –Representation of Graph –Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's - Single Source Shortest Path: Dijkstra's Algorithm.

Graph Terminologies

- A Graph is **a non-linear data structure** that consists of **vertices (nodes) and edges**.
- A vertex/node, is a point or an object in the Graph, and an edge is used to connect two vertices with each other.
- Graphs are non-linear because the data structure allows us to have different paths to get from one vertex to another, unlike with linear data structures like Arrays or Linked Lists.



- Formally, a graph is a pair of sets (V, E) , where V is the set of vertices and E is the set of edges, connecting the pairs of vertices. In the above graphs
- Eg: 1 set of vertices $V = \{ 1, 2, 3, 4, 5 \}$ and a set of edges $E = \{ (1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (4, 5) \}$.
- Eg2: **$V = \{ A, B, C, D, E, F, G \}$** and **$E = \{ (A, D), (A, C), (A, E), (E, C), (E, A), (C, G), (C, B), (C, F), (B, F) \}$**

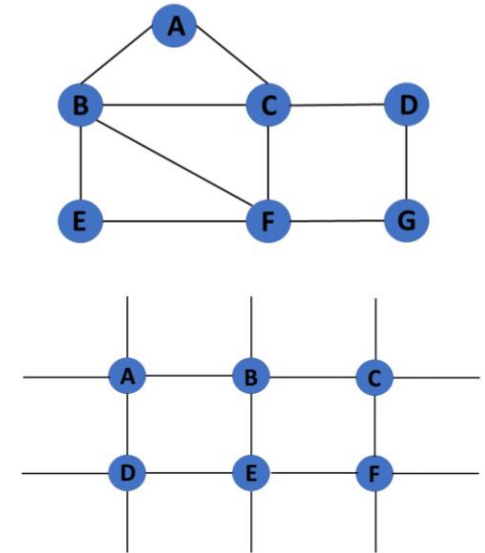
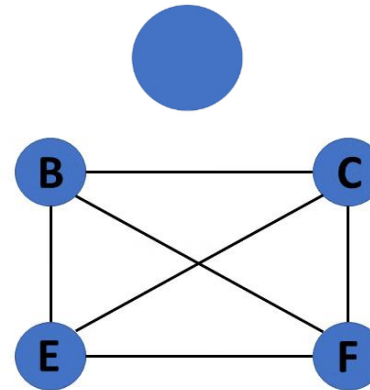
Types of Graphs

1. **Finite Graph** - if the number of vertices and edges in the graph is limited in number

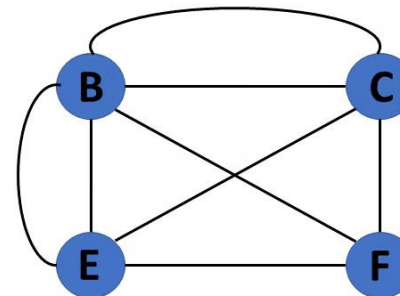
2. **Infinite Graph**- if the number of vertices and edges in the graph is interminable.

3. **Trivial Graph** - if it contains only a single vertex and no edges.

4. **Simple Graph** - If each pair of nodes or vertices in a graph $G=(V, E)$ has only one edge, it is a simple graph.

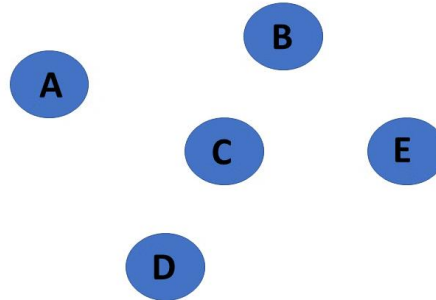


5. **Multi Graph** - If there are numerous edges between a pair of vertices in a graph $G= (V, E)$, the graph is referred to as a multigraph. There are no self-loops in a Multigraph.

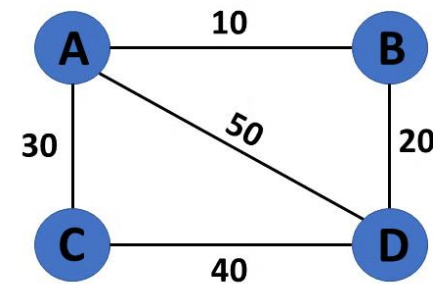


Types of Graphs

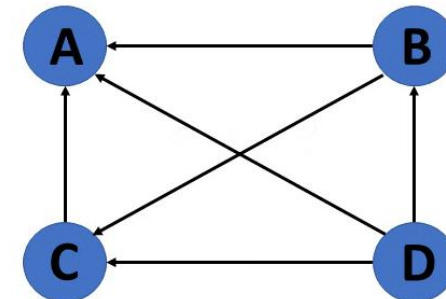
6. Null Graph - If several vertices but no edges connect them, a graph $G = (V, E)$ is a null graph.



7. Weighted Graph - A graph $G = (V, E)$ is called a labeled or weighted graph because each edge has a value or weight representing the cost of traversing that edge.

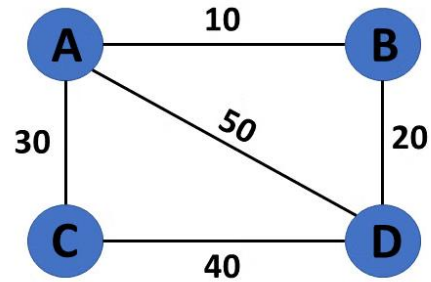


8. Directed Graph- A directed graph also referred to as a digraph, is a set of nodes connected by edges, each with a direction.

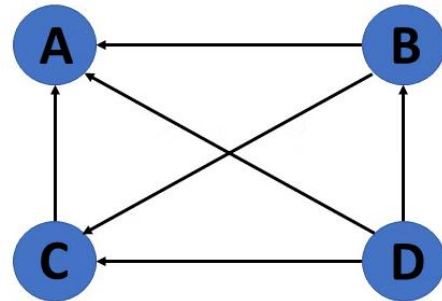


Types of Graphs

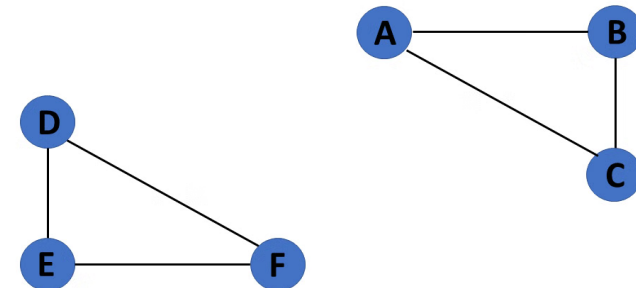
9. Undirected Graph- An undirected graph comprises a set of nodes and links connecting them, but with no direction of the edges.



10. Connected Graph- If there is a path between one vertex of a graph data structure and any other vertex, the graph is connected.

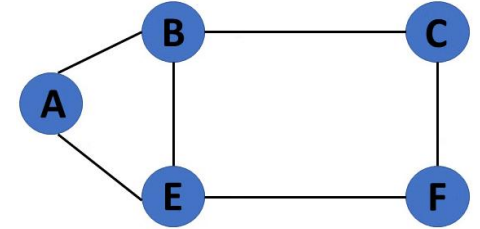


11. Disconnected Graph - When there is no edge linking the vertices, you refer to the null graph as a disconnected graph.

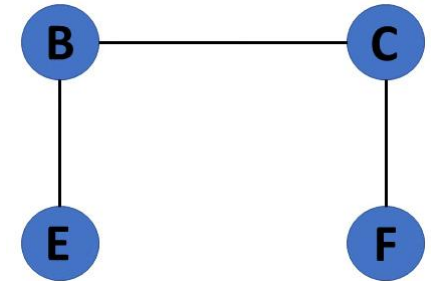


Types of Graphs

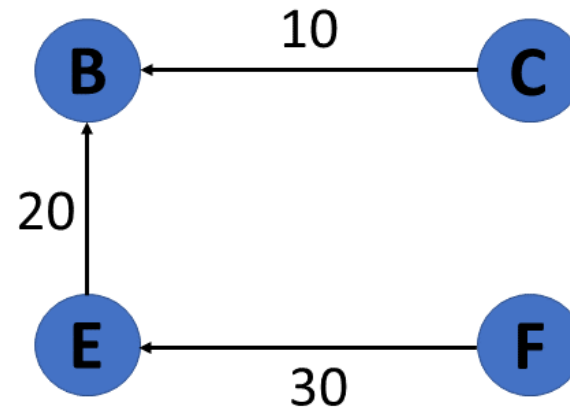
12. Cyclic Graph - If a graph contains at least one graph cycle, it is considered to be cyclic.



13. Acyclic Graph - When there are no cycles in a graph, it is called an acyclic graph.



14. Directed Acyclic Graph.- It's also known as a directed acyclic graph (DAG), and it's a graph with directed edges but no cycle. It represents the edges using an ordered pair of vertices since it directs the vertices and stores some data



Terminologies of Graphs

- An **edge** is one of the two primary units used to form graphs. Each edge has two ends, which are vertices to which it is attached
- If two vertices are endpoints of the same edge, they are **adjacent**
- A vertex's **outgoing edges** are directed edges that point to the origin
- A vertex's **incoming edges** are directed edges that point to the vertex's destination
- The **total number of edges** occurring to a vertex in a graph is its degree
- The **out-degree of a vertex** in a directed graph is the total number of outgoing edges, whereas the in-degree is the total number of incoming edges
- A vertex with an **in-degree of zero** is referred to as a **source vertex**, while one with an **out-degree of zero** is known as **sink vertex**
- An isolated vertex is a zero-degree vertex that is not an edge's endpoint

Terminologies of Graphs

- A **path** is a set of alternating vertices and edges, with each vertex connected by an edge
- **The path that starts** and finishes at the same vertex is known as a **cycle**
- A **path with unique vertices** is called a **simple path**.
- For each pair of vertices x, y , a graph is **strongly connected** if it contains a directed path from x to y and a directed path from y to x
- A directed graph is **weakly connected** if all of its directed edges are replaced with undirected edges, resulting in a connected graph. A weakly linked graph's vertices have at least one out-degree or in-degree
- **Forest** is a graph without a cycle.
- A **tree is a connected forest**. The primary form of the tree is called a **rooted tree**, which is a free tree.
- A **spanning subgraph** that is also a tree is known as a spanning tree.

Summary - Terminologies of Graphs

Term	Definition	Example
Vertex (Node)	A fundamental part of a graph representing an object or point.	In a social network graph, each person is a vertex.
Edge (Link)	A connection between two vertices in a graph.	In a social network graph, a friendship between two people is an edge.
Degree	The number of edges connected to a vertex.	In a social network, if a person has three friends, their degree is 3.
In-Degree	The number of incoming edges to a vertex in a directed graph.	In a Twitter network, the number of followers a person has.
Out-Degree	The number of outgoing edges from a vertex in a directed graph.	In a Twitter network, the number of people a person follows.
Path	A sequence of vertices connected by edges.	In a city map graph, a path represents the route from one city to another through connecting roads.
Cycle	A path that starts and ends at the same vertex without repeating any edges or vertices.	In a road network, traveling from city A to B to C and back to A forms a cycle.
Adjacency	Two vertices are adjacent if they are connected by an edge.	In a computer network, two computers directly connected by a cable are adjacent.

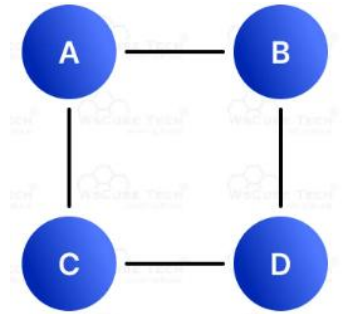
Representation of Graphs

Graphs can be represented in several ways. The three most common methods are

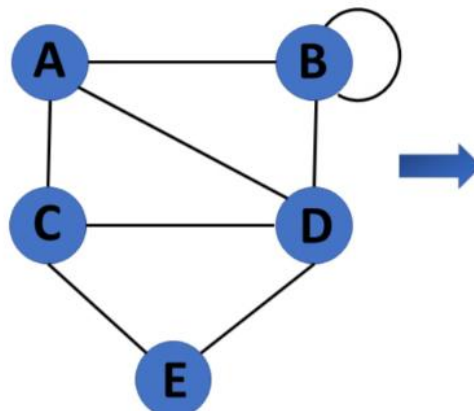
1. adjacency matrix, 2. adjacency list, 3. edge list.

1. Adjacency Matrix

- ❖ An adjacency matrix is a 2D array used to represent a graph.
- ❖ Each cell in the matrix indicates whether an edge exists between a pair of vertices.
- ❖ If an edge exists, the cell contains a value (typically 1 for unweighted graphs or the weight of the edge for weighted graphs); otherwise, it contains 0.
- ❖ You create an $M \times M$ matrix G for this representation. If an edge exists between vertex a and vertex b , the corresponding element of G , $g_{i,j} = 1$, otherwise $g_{i,j} = 0$
- **Example:** Consider a graph with 4 vertices (A, B, C, and D) and the following edges: A-B, A-C, B-D, and C-D.



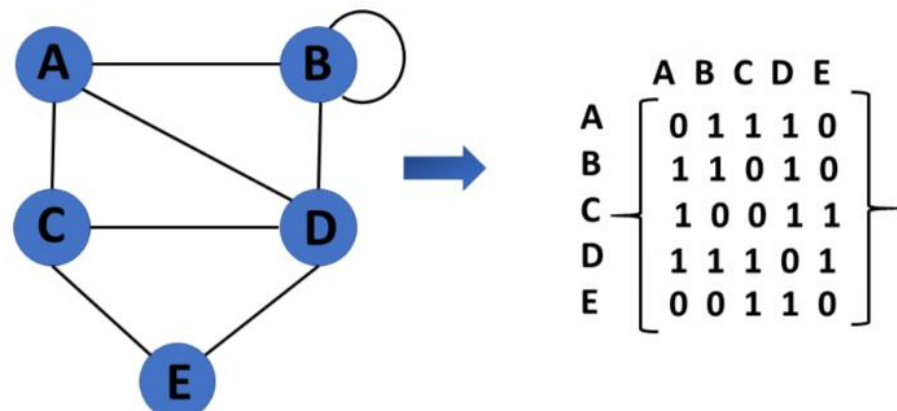
	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0



	A	B	C	D	E
A	0	1	1	1	0
B	1	1	0	1	0
C	1	0	0	1	1
D	1	1	1	0	1
E	0	0	1	1	0

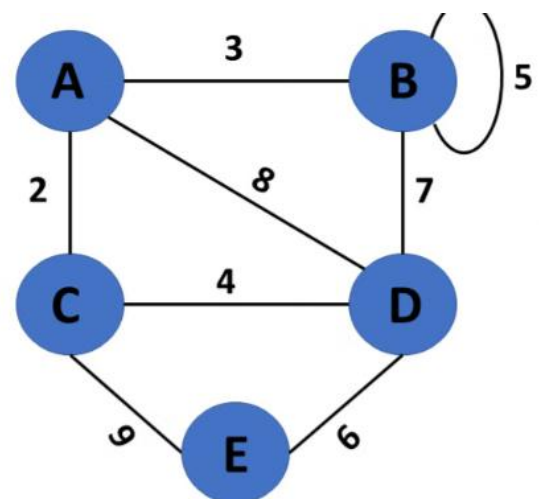
Representation of Graphs

Undirected Graph Representation



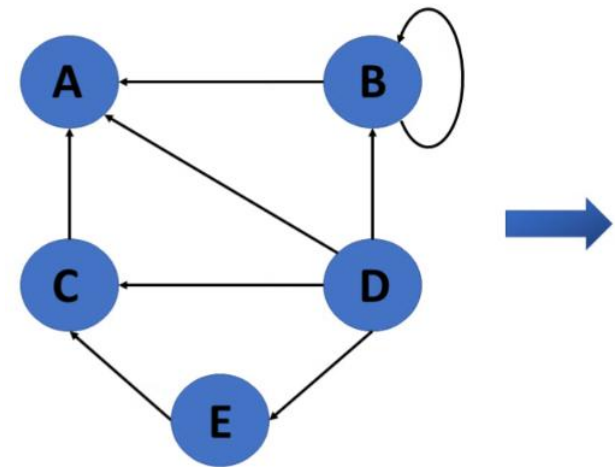
	A	B	C	D	E
A	0	1	1	1	0
B	1	1	0	1	0
C	1	0	0	1	1
D	1	1	1	0	1
E	0	0	1	1	0

Weighted Undirected Graph Representation

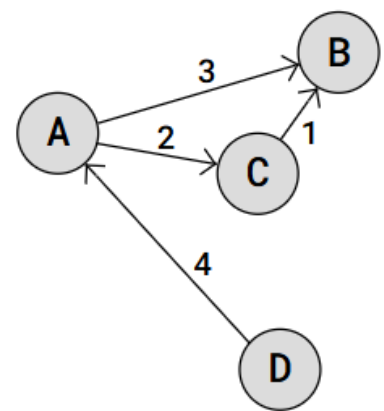


	A	B	C	D	E
A	0	3	2	8	0
B	3	5	0	7	0
C	2	0	0	4	9
D	8	7	4	0	6
E	0	0	9	6	0

Directed Graph Representation



	A	B	C	D	E
A	0	0	0	0	0
B	1	1	0	0	0
C	1	0	0	0	0
D	1	1	1	0	1
E	0	0	1	0	0



	A	B	C	D
A		3	2	
B				
C		1		
D	4			

In the adjacency matrix above, the value **3** on index **(0,1)** tells us there is an edge from vertex A to vertex B, and the weight for that edge is **3**.

Representation of Graphs-

2. Adjacency List

- An adjacency list is an array of lists.
- The array's indices represent the vertices, and each element in the array is a list of vertices adjacent to the vertex at that index.

Example:

Consider the same graph with 4 vertices (A, B, C, and D) and the following edges: A-B, A-C, B-D, and C-D.

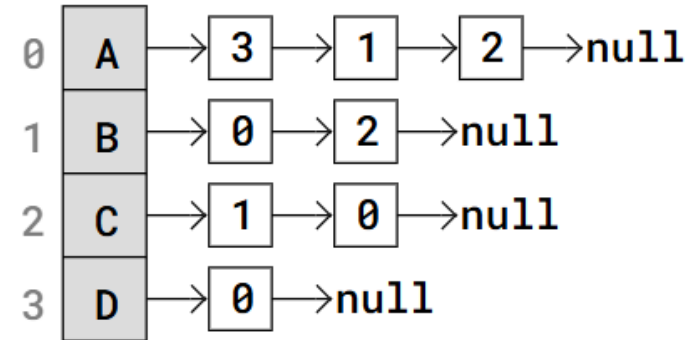
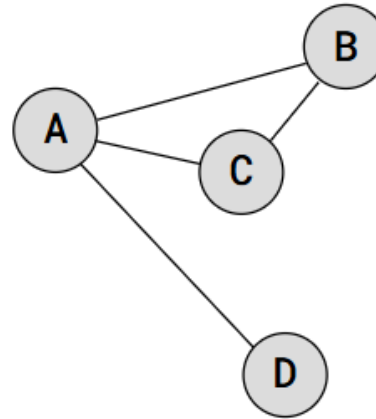
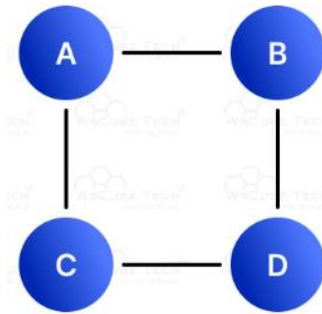
Adjacency List:

A: B, C

B: A, D

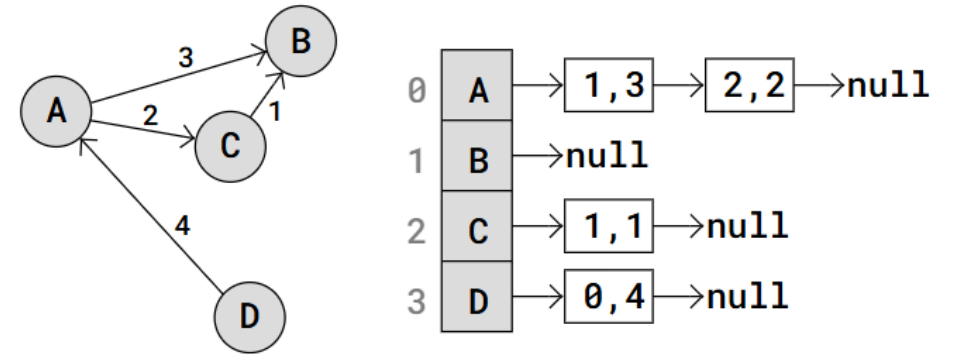
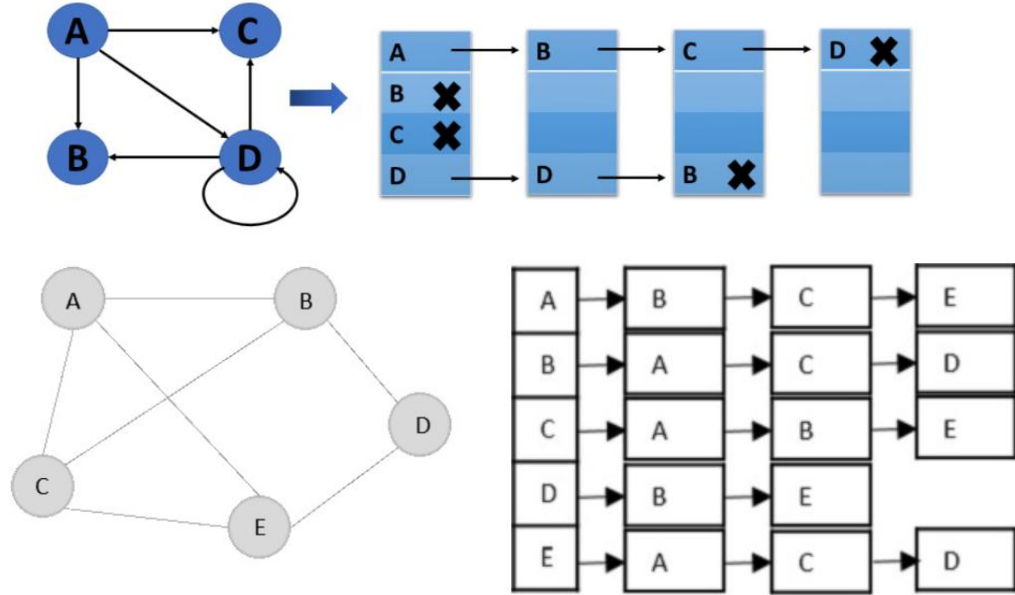
C: A, D

D: B, C



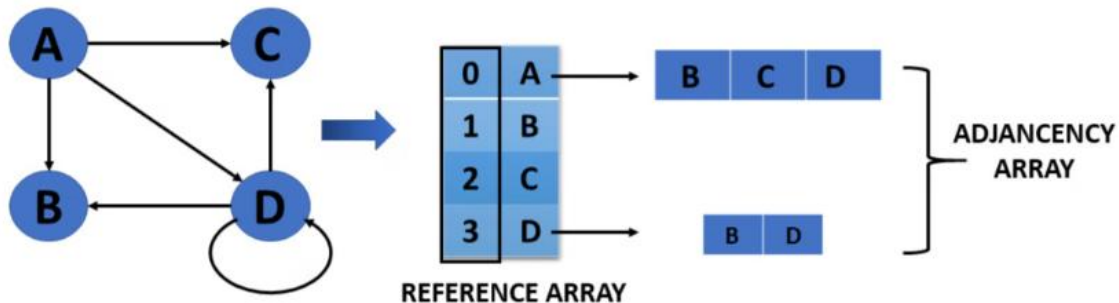
Adjacency List

Weighted Undirected Graph Representation Using Linked-List



A directed and weighted Graph and its adjacency list.

Weighted Undirected Graph Representation Using an Array



Operations on Graphs

The operations you perform on the graphs in data structures are listed below:

- Creating graphs
- Insert vertex
- Delete vertex
- Insert edge
- Delete edge

Creation of Graphs

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX_VERTICES 100
```

```
// Represents the directed graph using a 2D array
```

```
int  
adjacencyMatrix[MAX_VERTICES][MAX_VERTICES];  
int numVertices;
```

```
// Function to initialize the graph
```

```
void initGraph(int vertices) {  
    numVertices = vertices;  
    int i, j;  
    for (i = 0; i < numVertices; i++) {  
        for (j = 0; j < numVertices; j++) {  
            adjacencyMatrix[i][j] = 0;  
        }  
    }  
}
```

Creation of Graphs

// Function to add a directed edge from src to dest

```
void addEdge(int src, int dest) {  
    if (src >= 0 && src < numVertices && dest >= 0 && dest  
< numVertices) {  
        adjacencyMatrix[src][dest] = 1;  
    }  
}
```

// Function to print the adjacency matrix

```
void printGraph() {  
    int i, j;  
    printf("Adjacency Matrix:\n");  
    for (i = 0; i < numVertices; i++) {  
        for (j = 0; j < numVertices; j++) {  
            printf("%d ", adjacencyMatrix[i][j]);  
        }  
        printf("\n");  
    }  
}
```

```
int main() {  
    int num_v = 4;  
    initGraph(num_v);  
  
    // Add directed edges  
    addEdge(0, 1);  
    addEdge(0, 2);  
    addEdge(1, 2);  
    addEdge(2, 0);  
    addEdge(2, 3);  
    printGraph();  
    return 0;  
}
```


Implementation using an adjacency list

// A structure to represent an adjacency list node

(a) Adjacency List Node

```
struct AdjListNode {  
    int dest;  
    struct AdjListNode* next;  
};
```

Represents a node in a linked list.

➤ **dest** → destination vertex connected to the current vertex.

➤ **next** → pointer to the next adjacency node

Eg: If vertex 0 connects to 1 and 4, then adjacency list for vertex 0 → 1 → 4.

(b) Adjacency List

```
struct AdjList {  
    struct AdjListNode* head;  
};
```

➤ Each vertex in the graph has one adjacency list.

➤ head points to the first node in that list.

// A structure to represent the adjacency list

```
struct Graph {  
    int V;  
    struct AdjList* array;  
};
```

//V → total number of vertices.

//array → array of adjacency lists of size V.

array[0] → adjacency list of vertex 0

array[1] → adjacency list of vertex 1

Implementation using an adjacency list

// Function to Create a New Node

```
struct AdjListNode* newAdjListNode(int dest) {  
    struct AdjListNode* newNode = (struct AdjListNode*) malloc(sizeof(struct AdjListNode));  
    newNode->dest = dest;  
    newNode->next = NULL;  
    return newNode;  
}
```

- Creates and returns a new adjacency node.
- Stores the destination vertex.
- Initializes the next pointer as NULL.

Implementation using an adjacency list

// Function to create a graph with a given number of vertices

```
struct Graph* createGraph(int V) {  
    struct Graph* graph = (struct Graph*) malloc(sizeof(struct Graph));  
    graph->V = V;  
  
    graph->array = (struct AdjList*) malloc(V * sizeof(struct AdjList));  
  
    for (int i = 0; i < V; ++i)  
        graph->array[i].head = NULL;  
  
    return graph;  
}
```

- Allocates memory for the graph.
- Initializes each vertex's adjacency list as empty (NULL).
- Returns a pointer to the created graph.

Implementation using an adjacency list

// Function to add a directed edge to the graph (from src to dest)

```
void addEdge(struct Graph* graph, int src, int dest) {
```

```
    // Add node to source's list
```

```
    struct AdjListNode* newNode = newAdjListNode(dest);
```

```
    newNode->next = graph->array[src].head;
```

```
    graph->array[src].head = newNode;
```

```
    // Add node to destination's list (since undirected)
```

```
    newNode = newAdjListNode(src);
```

```
    newNode->next = graph->array[dest].head;
```

```
    graph->array[dest].head = newNode;
```

```
}
```

- Adds an edge between two vertices (src and dest).
- Since the graph is undirected, an edge is added both ways:
 - src $\rightarrow$ dest
 - dest $\rightarrow$ src
- Example: If addEdge(graph, 0, 1) is called:
 - 0 $\rightarrow$ 1
 - 1 $\rightarrow$ 0

Implementation using an adjacency list

// Function to print the graph

```
void printGraph(struct Graph* graph) {  
    for (int v = 0; v < graph->V; ++v) {  
        struct AdjListNode* temp = graph->array[v].head;  
        printf("\n Adjacency list of vertex %d\n head ", v);  
        while (temp) {  
            printf("-> %d", temp->dest);  
            temp = temp->next;  
        }  
        printf("\n");  
    }  
}
```

- Traverses each adjacency list.
- Prints all vertices connected to each vertex.

Implementation using an adjacency list

```
int main() {  
    int V = 5;  
    struct Graph* graph = createGraph(V);  
    addEdge(graph, 0, 1);  
    addEdge(graph, 0, 4);  
    addEdge(graph, 1, 2);  
    addEdge(graph, 1, 3);  
    addEdge(graph, 1, 4);  
    addEdge(graph, 2, 3);  
    addEdge(graph, 3, 4);  
  
    printGraph(graph);  
    return 0;  
}
```

- Creates a graph with 5 vertices (0–4).
- Adds edges between them.
- Finally prints the adjacency list.

➤ Output:

Adjacency list of vertex 0
head -> 4 -> 1

Adjacency list of vertex 1
head -> 4 -> 3 -> 2 -> 0

Adjacency list of vertex 2
head -> 3 -> 1

Adjacency list of vertex 3
head -> 4 -> 2 -> 1

Adjacency list of vertex 4
head -> 3 -> 1 -> 0